Оптимизация Экономики и Контекста LLM в Средах Разработки: Глубокий Анализ Архитектурных и Инженерных Решений

Введение

В современной индустрии разработки программного обеспечения (SDLC) происходит фундаментальный сдвиг парадигмы: переход от изолированных инструментов автодополнения кода к интегрированным агентным системам на базе больших языковых моделей (LLM). Инструменты нового поколения, такие как Cursor, Google Antigravity, GitHub Copilot и JetBrains AI Assistant, трансформируют IDE из текстовых редакторов в центры управления автономными цифровыми сотрудниками. Однако этот технологический скачок привел к возникновению критической проблемы: экспоненциальному росту операционных расходов, связанных с потреблением токенов.

В сценариях, где агент должен «прочитать» весь проект для внесения изменения в одну функцию, стоимость одной транзакции может достигать значений, делающих масштабное внедрение ИИ экономически нецелесообразным без строгой оптимизации. Эффективность использования токенов (Token Efficiency) становится ключевой метрикой инженерной продуктивности.

Данный отчет представляет собой исчерпывающее исследование методов минимизации расхода токенов и оптимизации контекста. Мы систематизируем архитектурные, процессные и инженерные подходы, охватывая весь спектр решений — от низкоуровневых настроек IDE до высокоуровневых стратегий управления знаниями через Model Context Protocol (MCP).

Часть 1. Архитектура и Экономика Токенов в Агентных Системах

Понимание микроэкономики LLM является фундаментом для построения эффективных стратегий оптимизации. Стоимость взаимодействия с моделью не является линейной функцией от полезности ответа; она представляет собой сложную производную от объема контекста, стратегии кэширования и архитектуры выбранной модели.

1.1. Дисбаланс Входных и Выходных Потоков

В задачах разработки программного обеспечения наблюдается колоссальный дисбаланс между объемом входных данных (Input Tokens) и генерируемым результатом (Output Tokens). Для корректного изменения 10 строк кода агентной системе часто требуется загрузить в контекст тысячи строк связанных файлов, определений типов, документации и истории предыдущих изменений. Соотношение Input:Output может достигать 1000:1.

В экономических моделях большинства провайдеров (Anthropic, OpenAI, Google) входные токены дешевле выходных (обычно в соотношении 1:3 или 1:5). Однако, учитывая объем, именно входной контекст формирует до 90% стоимости сессии. Проблема усугубляется в агентных рабочих процессах, где каждое действие модели (рассуждение, вызов инструмента, чтение файла) добавляется в историю диалога, создавая эффект «снежного кома» или контекстной инфляции.[1] Без механизмов очистки или кэширования стоимость каждого последующего шага агента растет линейно, что делает длительные сессии рефакторинга запретительно дорогими.

1.2. Технологии Кэширования Контекста (Context Caching)

Революцией 2024–2025 годов стало внедрение механизмов кэширования контекста на уровне API, что позволяет изменить экономику взаимодействия с LLM.

1.2.1. Механика Anthropic Prompt Caching

Anthropic реализует подход, основанный на префиксном кэшировании. Система анализирует входящий промпт и, если его начало совпадает с ранее обработанным запросом, использует предварительно вычисленные активации модели, вместо того чтобы обрабатывать токены заново.

Архитектурные особенности:

- **Гранулярность:** Кэширование применяется к блокам, явно помеченным параметром cache_control. Кэшируется полный префикс до указанного блока включительно.
- **Время жизни (TTL):** По умолчанию кэш живет 5 минут. Однако каждый раз, когда кэшированный контент используется (cache hit), таймер сбрасывается, продлевая жизнь кэша еще на 5 минут без дополнительных затрат. Это делает модель идеальной для активных сессий кодинга в IDE, где запросы следуют один за другим с высокой частотой.[3]
- **Экономика:** Запись в кэш стоит дороже базовых входных токенов (+25%), но чтение из кэша дешевле на 90%. Точка безубыточности достигается уже при втором обращении к контексту.

1.2.2. Механика Google Gemini Context Caching

Google предлагает альтернативную модель, ориентированную на сверхдлинные

контексты (до 2 миллионов токенов) и более длительное хранение.

Архитектурные особенности:

- **Явное создание:** Кэш создается как отдельный ресурс с уникальным идентификатором.
- **TTL:** По умолчанию 1 час, с возможностью продления. Это позволяет хранить статический контекст (например, полную документацию фреймворка, весь репозиторий кода или корпоративную базу знаний) в течение длительного времени, даже если разработчик делает перерывы в работе.[5]
- **Implicit Caching:** В новых моделях (Gemini 2.5) внедрено неявное кэширование, которое автоматически оптимизирует повторяющиеся префиксы без явного создания ресурсов, возвращая пользователю 75% скидку на токены.[7]

Сравнительный анализ моделей кэширования

Характеристика	Anthropic (Claude)	Google (Gemini)
Механизм активации	cache_control в заголовках	Explicit Resources / Implicit
Базовый TTL	5 минут (обновляемый)	1 час (продляемый)
Стоимость записи	1.25x от базовой цены	Стандартная цена
Стоимость чтения	0.1x от базовой цены (скидка 90%)	Скидка 75-90% + аренда/час
Сценарий использования	Активные сессии рефакторинга, Chat-loop	Долгоживущие агенты, RAG, анализ репозиториев

1.3. Принципиальная Схема Оптимизации

На основе анализа архитектуры моделей можно выделить три фундаментальных направления оптимизации, которые должны применяться каскадно:

1. **Фильтрация (Filtering):** Агрессивное исключение нерелевантной информации до того, как она попадет в контекстное окно. Это реализуется через настройки IDE (.cursorignore, .aiignore) и управление областью видимости.
2. **Сжатие (Compression):** Использование форматов данных с высокой семантической плотностью. Замена полных файлов на Unified Diffs, удаление избыточного

форматирования, использование техник типа LLMLingua для компрессии промптов.
3. **Кэширование (Caching):** Максимизация повторного использования статических частей промпта (системные инструкции, правила проекта, документация) через механизмы провайдеров.

Часть 2. Настройки IDE и Инструментария: Тонкая Настройка Среды

Среда разработки (IDE) выступает в роли шлюза, контролирующего поток информации к модели. Правильная конфигурация IDE способна сократить потребление токенов на 40-60% без изменения привычек разработчика.

2.1. Cursor: Флагман AI-Разработки

Cursor предоставляет наиболее глубокие возможности по управлению контекстом, но настройки по умолчанию часто оптимизированы для удобства, а не для экономии.

2.1.1. Стратегия Игнорирования: .cursorignore vs .cursorindexingignore

Многие разработчики ошибочно полагают, что файл .gitignore достаточен для управления контекстом ИИ. Однако Cursor индексирует локальную базу кода для RAG (Retrieval-Augmented Generation), и мусорные файлы, игнорируемые Git, могут попадать в индекс, если не настроены специально.

- **.cursorignore:** Этот файл работает как жесткий блокировщик. Файлы, перечисленные здесь, не только исключаются из индексации, но и не могут быть добавлены в контекст чата даже вручную.
  - *Рекомендация:* Сюда необходимо добавлять файлы package-lock.json, yarn.lock, бинарные файлы, большие сгенерированные JSON-датасеты, файлы логов и директории сборки (dist/, build/). Попадание файла блокировки зависимостей (lockfile) в контекст может мгновенно израсходовать десятки тысяч токенов без какой-либо пользы для генерации логики.[8]
- **.cursorindexingignore:** Файлы, указанные здесь, исключаются из автоматического векторного поиска (Codebase Search), но остаются доступными для ручного добавления через @File.
  - *Сценарий:* Устаревшие модули (legacy code), которые могут сбить агента с толку при поиске ответов на вопросы о новой архитектуре, но которые могут понадобиться для точечного рефакторинга. Исключение их из индекса повышает релевантность ответов (Recall) и снижает затраты на поиск.[9]
  - *Безопасность:* Исключение файлов с секретами или конфигурационных файлов через эти настройки критически важно не только для экономии, но и для предотвращения утечек данных.[10]

2.1.2. Режимы Работы: Fast Apply и Speculative Decoding

Cursor внедрил технологию, которая меняет экономику редактирования кода — **Fast Apply** (иногда называемую "Instant Apply" или "Yolo Mode").

- **Архитектура:** Вместо того чтобы использовать большую и дорогую модель (например, Claude 3.5 Sonnet) для генерации полного текста измененного файла, Cursor использует гибридный подход. "Умная" модель генерирует план изменений, а специализированная малая модель (custom speculative model) применяет эти изменения к коду с огромной скоростью (~1000 токенов в секунду).
- **Влияние на токены:** Помимо снижения задержки (latency), этот подход снижает количество *входных* токенов для последующих запросов, так как история чата не засоряется длинными листингами кода и неудачными попытками генерации.
- **Speculative Edits:** Технология спекулятивного декодирования позволяет модели "предсказывать" следующие токены группами, что значительно удешевляет процесс вывода по сравнению с полным авторегрессионным циклом.[11]

2.1.3. Управление Контекстом в Чате

- **Авто-контекст:** По умолчанию Cursor пытается "угадать" релевантные файлы. В больших проектах это часто приводит к "галлюцинациям релевантности", когда в контекст подтягиваются файлы с похожими названиями, но из других модулей. Опытные пользователи отключают авто-контекст и используют явное добавление файлов (Cmd+Enter с выбранными файлами) или @Symbols.
- **Сброс сессии:** Cursor не хранит состояние между чатами на сервере таким образом, чтобы экономить токены. Каждое сообщение в длинном треде отправляет всю историю заново. Регулярное использование Cmd+L (New Chat) — самый простой способ сброса счетчика токенов.[2]

2.2. Google Antigravity: Агент-ориентированная Парадигма

Google Antigravity вводит концепцию разделения ролей между "Менеджером" и "Исполнителем", что открывает новые возможности для оптимизации.

2.2.1. Planning Mode vs. Execution Mode

Ключевая инновация Antigravity — режим планирования (**Planning Mode**). В этом режиме агент (использующий Gemini 3 Pro) анализирует задачу и строит план действий *без* написания кода.

- **Экономия:** План представляет собой текстовое описание на естественном языке, которое занимает в разы меньше токенов, чем код. Пользователь валидирует план, и только после утверждения агент переходит в режим исполнения (**Execution Mode**), где генерирует код. Это предотвращает дорогостоящие ошибки, когда агент пишет сотни строк кода "не туда", тратя квоту впустую.[12]
- **Асинхронность:** "Manager Surface" позволяет запускать агентов в фоне. Поскольку

оплата (или квота) часто привязана к времени вычислений или токенам, возможность агента работать автономно и останавливаться при достижении цели (или лимита попыток) критична.

2.2.2. Конфигурация Агентов

Antigravity использует файлы конфигурации для управления поведением агентов, аналогичные .cursorrules, но с большим упором на права доступа.

- **Terminal Policy:** Настройка Auto позволяет агенту выполнять команды терминала (например, grep, ls) самостоятельно. Это позволяет агенту "исследовать" среду, получая компактный вывод терминала, вместо того чтобы запрашивать чтение полных файлов. Эффективное использование терминальных команд для разведки (reconnaissance) может снизить объем потребляемого контекста на порядок.[14]

2.3. VS Code Copilot и JetBrains AI

Традиционные IDE также адаптируются к требованиям экономии, хотя и с меньшей гибкостью.

2.3.1. VS Code Copilot: Instruction Only Mode

В VS Code существует скрытая жемчужина оптимизации — режим работы без контекста или с минимальным контекстом.

- **Temporal Context:** Copilot по умолчанию отправляет в модель информацию о недавно открытых и отредактированных файлах. Это часто избыточно. Настройка github.copilot.chat.edits.temporalContext.enabled: false отключает это поведение, заставляя модель работать только с тем, что явно предоставлено.[15]
- **Instruction Only:** Для задач генерации нового кода (boilerplate), написания документации или объяснения алгоритмов контекст проекта часто не нужен вовсе. Использование чистого режима (без @workspace) позволяет генерировать код, используя только знания самой модели, что экономит токены и ускоряет ответ.[16]

2.3.2. JetBrains AI Assistant: AI Credits и .aiignore

JetBrains использует модель оплаты через AI Credits, что делает оптимизацию токенов напрямую эквивалентной экономии денег.

- **.aiignore:** Аналог .cursorignore. Критически важен для Java/Kotlin проектов, где папки target/, .gradle/ и build/ могут содержать гигабайты текста, который IDE может попытаться проиндексировать. Корректная настройка .aiignore предотвращает утечку кредитов на обработку бинарного мусора.[18]
- **Выбор модели:** JetBrains позволяет переключаться между моделями. Для простых задач (генерация unit-тестов, документация) использование Gemini Flash вместо GPT-4o расходует значительно меньше кредитов при приемлемом качестве.[20]

Часть 3. Промпт-инжиниринг: От Текста к Инженерной Дисциплине

В эпоху агентных систем промпт-инжиниринг трансформируется из искусства формулирования вопросов в инженерную дисциплину управления системными инструкциями и контекстом.

3.1. .cursorrules: Конституция Проекта

Файл .cursorrules (и его аналоги, такие как AGENTS.md в Antigravity) автоматически добавляется в системный промпт каждого запроса. Это делает его самым мощным рычагом воздействия на поведение агента и потребление токенов.

3.1.1. Стратегии Минимизации Вывода (Output Token Reduction)

Выходные токены — самый дорогой ресурс. .cursorrules должен содержать жесткие инструкции по сокращению многословия.

- **Запрет на "болтовню":** Модели обучены быть вежливыми и многословными ("Certainly! Here is the code you requested..."). В масштабах года работы команды эти фразы складываются в миллионы токенов.
  - *Правило:* NO conversational filler. Output ONLY the code block or the diff. Be concise..[21]
- **Принудительный формат Diff:** Вместо того чтобы переписывать файл целиком, модель должна возвращать только изменения.
  - *Правило:* When editing files, ALWAYS use Unified Diff format. Do NOT rewrite the whole file unless >50% changed.
  - *Эффект:* Для файла в 500 строк изменение одной функции займет 20 токенов (diff) вместо 1500 токенов (полный файл).[23]

3.1.2. Zero-Shot Referencing и Модульность Правил

Классический промпт-инжиниринг рекомендует Few-Shot Learning (давать примеры). В контексте оптимизации токенов это дорого, так как примеры потребляют входной контекст при *каждом* запросе.

- **Zero-Shot Referencing:** Вместо вставки текста примера в .cursorrules, следует давать ссылку на файл в проекте, который является эталоном.
  - *Пример:* For authentication logic, refer to patterns in @src/auth/auth.service.ts. Агент прочитает этот файл только при необходимости (lazy loading) или использует его из кэша.
- **Модульность:** Вместо одного гигантского файла правил, эффективно использовать структуру с условной загрузкой (glob patterns).
  - .cursor/rules/frontend.mdc (загружается только для *.tsx)

- .cursor/rules/backend.mdc (загружается только для *.go)
  Это позволяет держать системный промпт компактным и релевантным текущему контексту.22

3.2. Парадокс "Ленивого Кодинга" и Guardrails

Модели GPT-4 Turbo и 4o склонны к "лени" — они возвращают код с комментариями //... existing code.... Это кажется экономией токенов, но на практике приводит к их перерасходу, так как разработчик вынужден писать дополнительные промпты ("Напиши код полностью", "Ты пропустил этот кусок"), перезапускать генерацию и тратить время.

- **Guardrails против лени:** В системный промпт необходимо внедрить инструкции, которые балансируют между краткостью и полнотой.
  - *Инструкция:* If the file is large (>300 lines), output a Unified Diff. If the file is small, output the full file. Never leave placeholders like '//... rest of code' inside the modified function blocks.
  - Aider демонстрирует, что использование Unified Diff формата снижает "лень" модели в 3 раза по сравнению с блоками SEARCH/REPLACE, так как формат diff более строгий и алгоритмически понятный для модели.[25]

3.3. Технологии Сжатия Промптов: LLMLingua

Для продвинутых сценариев, особенно при построении собственных инструментов поверх API (Build-your-own-agent), применяются алгоритмические методы сжатия.

- **LLMLingua:** Библиотека от Microsoft, использующая малые модели (например, Llama-2-7b или BERT) для вычисления перплексии токенов в промпте. Токены с низкой перплексией (предсказуемые, несущие мало информации) удаляются.
- **Результат:** Возможно сжатие промпта на 20-40% с сохранением семантического смысла и ключевых инструкций. Это особенно эффективно для сжатия длинной технической документации перед подачей её в контекст агента.[26]

Часть 4. Работа с Кодом: Диффы, Локализация и Форматы

Способ взаимодействия с кодом определяет не только удобство, но и "вес" транзакции в токенах.

4.1. Битва Форматов: Unified Diff vs. Whole File

Выбор формата выходных данных является критическим решением.

Формат	Описание	Расход Токенов	Риски и

		(Output)	Особенности
Whole File Rewrite	Модель возвращает весь файл целиком после изменений.	**Максимальный.** Для файла в 1000 строк изменение 1 строки стоит 1000+ токенов.	Высокая надежность (легко применить), но крайне дорого и медленно. Высокий риск обрыва генерации по таймауту.
Search/Replace Block	Блоки <<<< SEARCH... REPLACE >>>>.	**Средний.** Зависит от уникальности заменяемого блока.	Модель часто ошибается в точном цитировании блока поиска (пробелы, отступы), что приводит к сбою применения патча.
Unified Diff	Стандартный формат diff (@@ -10,4 +10,5 @@).	**Минимальный.** Возвращаются только измененные строки + контекст (3 строки).	Требует высокой "интеллектуальности" модели для подсчета строк. Идеально для GPT-4o/Claude 3.5 Sonnet.

Анализ Aider:
Инструмент Aider провел обширные бенчмарки и выяснил, что формат Unified Diff является оптимальным для современных сильных моделей. Он не только экономит токены, но и повышает качество кода, так как модель "видит", что она меняет, в строгом формате. Для слабых моделей этот формат не подходит, так как они теряются в подсчете номеров строк.25

4.2. Локализация Изменений и Inline Instructions

В IDE (VS Code, JetBrains, Cursor) существуют механизмы Inline Chat (вызов ИИ прямо в редакторе).

- **Scoped Context:** Когда разработчик выделяет метод и нажимает Cmd+I (или Cmd+K

в Cursor), IDE отправляет в LLM только выделенный фрагмент и минимально необходимый окружающий контекст. Это радикально дешевле, чем отправка всего файла в боковую панель чата.
- **Стратегия:** Для точечных изменений (рефакторинг функции, добавление документации, исправление бага) всегда использовать Inline-режим с выделением. Боковую панель (Chat Panel) оставлять только для высокоуровневых вопросов ("Как архитектурно связать модуль А и Б?").[30]

Часть 5. Инструменты и MCP: Архитектура Подключаемых Знаний

Model Context Protocol (MCP) — это открытый стандарт для подключения внешних данных к LLM. Однако неправильное использование MCP является главной причиной "раздувания" контекста (Context Bloat).

5.1. Проблема MCP Bloat

При подключении инструментов (например, Jira, PostgreSQL, GitHub) их определения (JSON Schema) добавляются в системный промпт. Если подключено 20 инструментов, их описания могут занимать 5-10 тысяч токенов *в каждом запросе*. Это создает постоянную "налоговую нагрузку" на контекст, даже если инструменты не используются.

5.2. Решение: Динамическая Загрузка Инструментов (Dynamic Toolsets)

Для решения этой проблемы был разработан паттерн **Tool Discovery** (предложен Anthropic и реализован в платформах типа Speakeasy).

1. **Registry Tool:** Вместо загрузки всех 50 инструментов, в контекст агента загружается только *один* инструмент поиска: search_tools(query).
2. **On-Demand Loading:** Когда агент понимает, что ему нужно работать с базой данных, он вызывает search_tools("database").
3. **Injection:** Система находит релевантные инструменты (например, sql_query, get_table_schema) и динамически добавляет их определения в контекст *только на время текущей сессии* или даже текущего шага.
4. **Результат:** Снижение потребления входных токенов на 95% в сложных экосистемах инструментов. Это позволяет масштабировать количество доступных агенту инструментов до тысяч без деградации производительности и роста цены.[32]

5.3. Code Execution Environment (Песочницы)

Еще один мощный способ экономии — замена множества API-инструментов одним универсальным инструментом исполнения кода (Code Execution).

- **Сценарий:** Агенту нужно проанализировать CSV-файл на 100 МБ.
  - *API подход:* Агент читает файл по кускам (Chunking), отправляет их в LLM, просит проанализировать. Расход токенов огромен.
  - *Code Execution подход:* Агент пишет Python-скрипт (100 токенов) для анализа файла (используя Pandas), запускает его в песочнице и получает результат (например, "Среднее значение: 42").
- **Преимущество:** Обработка данных переносится с LLM (дорого) на CPU (дешево). Агент оперирует *логикой* обработки, а не самими данными. Anthropic активно продвигает этот подход в своих инструментах анализа.[35]

Часть 6. Процессные Приёмы, Политики и Культура

Технологическая оптимизация бессильна без соответствующей культуры использования.

6.1. Психология "Нового Чата" (The New Chat Discipline)

Самая простая и эффективная привычка, которую игнорируют большинство разработчиков — регулярный сброс контекста.

- **Проблема:** Разработчики склонны держать один бесконечный чат "на все случаи жизни". К концу дня этот чат может содержать 50k+ токенов истории. Каждый новый вопрос ("А как назвать переменную?") отправляет эти 50k токенов в модель.
- **Решение:** Внедрение культуры "Одна задача — Один чат". Современные IDE должны внедрять UX, поощряющий это (например, кнопка "Resolve & New Task", которая суммаризирует итоги текущего чата в память проекта и открывает чистый лист).[16]

6.2. Seat Management и Корпоративные Политики

Для организаций, использующих GitHub Copilot Business/Enterprise ($19-39/мес за место), критично управление лицензиями.

- **Dormant Users:** Использование API для отслеживания активности. Если разработчик не использовал Copilot 30 дней, лицензия должна быть автоматически отозвана.
- **Централизация Правил:** Использование возможности централизованного управления .cursorrules или политиками Copilot на уровне организации гарантирует, что все разработчики используют оптимизированные настройки (например, запрет на индексацию тестовых данных).[36]

Часть 7. Примеры и Сравнение (ROI)

7.1. Сценарий: Рефакторинг Legacy-модуля (Java, 50 файлов)

Сравним два подхода к решению задачи рефакторинга модуля.

Параметр	Подход А: Наивный (Chat-based)	Подход Б: Оптимизированный (Agentic + Caching)
Инструмент	Стандартный Web Chat / Copilot без настроек	Cursor + Anthropic Prompt Caching + Unified Diff
Действие	Copy-paste 5 файлов, промпт "Перепиши на новый стиль"	Промпт "Refactor @UserModule using patterns in @BaseModule. Output unified diffs."
Контекст (Input)	50k токенов (каждый запрос полный)	100k токенов (весь проект), но **кэшировано** (скидка 90% на чтение)
Вывод (Output)	10k токенов (полные файлы)	500 токенов (только диффы)
Стоимость итерации	~$1.50	~$0.15
Качество	Потеря контекста зависимостей, ручной copy-paste	Полный контекст проекта, автоматическое применение
Итоговый ROI	Базовый	**10x экономия + выше скорость**

7.2. Trade-offs: Когда Экономия Вредна

Оптимизация не должна идти в ущерб качеству.

1. **Чрезмерное сжатие:** Если через .ignore скрыть слишком много файлов, агент может не заметить зависимостей и сломать билд. Экономия на токенах приведет к затратам времени разработчика на отладку.
2. **Слабые модели:** Использование дешевых моделей (GPT-4o-mini, Gemini Flash) для сложных архитектурных задач часто приводит к генерации нерабочего кода, который приходится переделывать. Для сложных задач ("Deep Reasoning") всегда следует использовать топовые модели (Claude 3.5 Sonnet, GPT-4o, o1), но подавать им рафинированный, оптимизированный контекст. Экономить нужно на *мусоре* в

контексте, а не на *интеллекте* модели.[38]

Заключение

Минимизация стоимости и контекста LLM в разработке — это не разовая настройка, а непрерывный процесс, требующий комплексного подхода. Переход от наивного использования чат-ботов к инженерно-обоснованным агентным процессам включает в себя архитектурные решения (Prompt Caching, MCP Tool Discovery), тонкую настройку среды (.cursorrules, .ignore) и дисциплину работы с кодом (Unified Diffs).

Команды, успешно внедрившие эти практики, демонстрируют возможность снижения расходов на токены на 70-90% при одновременном повышении качества кода за счет предоставления моделям более чистого и релевантного контекста. В новой эре разработки контекст становится самым ценным ресурсом, и его эффективная переработка определяет конкурентное преимущество инженерной команды.

Источники

1. Understanding LLM Token Usage - How To - Cursor - Community Forum, дата последнего обращения: января 2, 2026, https://forum.cursor.com/t/understanding-llm-token-usage/120673
2. Understanding Cursor Token Usage: What I've Learned So Far - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/cursor/comments/1m29nf1/understanding_cursor_token_usage_what_ive_learned/
3. Prompt caching - Claude Docs, дата последнего обращения: января 2, 2026, https://platform.claude.com/docs/en/build-with-claude/prompt-caching
4. Prompt Caching with OpenAI, Anthropic, and Google Models - PromptHub, дата последнего обращения: января 2, 2026, https://www.prompthub.us/blog/prompt-caching-with-openai-anthropic-and-google-models
5. Context caching | Gemini API | Google AI for Developers, дата последнего обращения: января 2, 2026, https://ai.google.dev/gemini-api/docs/caching
6. Context caching overview | Generative AI on Vertex AI - Google Cloud Documentation, дата последнего обращения: января 2, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview
7. Gemini 2.5 Models now support implicit caching - Google Developers Blog, дата последнего обращения: января 2, 2026, https://developers.googleblog.com/en/gemini-2-5-models-now-support-implicit-caching/
8. Mastering .cursorignore, .cursor/rules, and @Docs in Cursor IDE | Swift Published, дата последнего обращения: января 2, 2026, https://swiftpublished.in/article/cursor-rules,-docs,-ignore
9. Ignore files | Cursor Docs, дата последнего обращения: января 2, 2026,

https://cursor.com/docs/context/ignore-files
10. Mastering Cursor IDE: 10 Best Practices (Building a Daily Task Manager App) - Medium, дата последнего обращения: января 2, 2026, https://medium.com/@roberto.g.infante/mastering-cursor-ide-10-best-practices-building-a-daily-task-manager-app-0b26524411c1
11. Editing Files at 1000 Tokens per Second - Cursor, дата последнего обращения: января 2, 2026, https://cursor.com/blog/instant-apply
12. Build with Google Antigravity, our new agentic development platform, дата последнего обращения: января 2, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
13. Antigravity: Google's NEW Agentic Editor in 7 Minutes, дата последнего обращения: января 2, 2026, https://www.youtube.com/watch?v=wbPpvjcAHew
14. How to Set Up and Use Google Antigravity - Codecademy, дата последнего обращения: января 2, 2026, https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity
15. Option at settings to not add opened file as context automatically. : r/GithubCopilot - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/GithubCopilot/comments/1okspdu/option_at_settings_to_not_add_opened_file_as/
16. What I've Learned About GitHub Copilot Agent Mode - DEV Community, дата последнего обращения: января 2, 2026, https://dev.to/anchildress1/what-ive-learned-about-github-copilot-agent-mode-4co2
17. Show HN: Stop Claude Code from forgetting everything | Hacker News, дата последнего обращения: января 2, 2026, https://news.ycombinator.com/item?id=46426624
18. Restrict or disable AI Assistant features - JetBrains, дата последнего обращения: января 2, 2026, https://www.jetbrains.com/help/ai-assistant/disable-ai-assistant.html
19. .aiignore | Junie Documentation - JetBrains, дата последнего обращения: января 2, 2026, https://www.jetbrains.com/help/junie/aiignore.html
20. How to control AI Assistant's quota usage to avoid high token consumption? - YouTrack, дата последнего обращения: января 2, 2026, https://youtrack.jetbrains.com/projects/GO/articles/SUPPORT-A-2448/How-to-control-AI-Assistants-quota-usage-to-avoid-high-token-consumption
21. How to Use Cursor More Efficiently! : r/ChatGPTCoding - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1hu276s/how_to_use_cursor_more_efficiently/
22. Rules | Cursor Docs, дата последнего обращения: января 2, 2026, https://cursor.com/docs/context/rules
23. RFC: Token-Efficient and Consistent AI Code Generation Through GNU Unified Diff, дата последнего обращения: января 2, 2026,

https://medium.com/@zackisland/rfc-token-efficient-and-consistent-ai-code-generation-through-gnu-unified-diff-md-a07f676c975a
24. Using nested JSON Skillsets to save 90% of tokens with Cursor, дата последнего обращения: января 2, 2026, https://forum.cursor.com/t/using-nested-json-skillsets-to-save-90-of-tokens-with-cursor/141399
25. Unified diffs make GPT-4 Turbo 3X less lazy - Aider, дата последнего обращения: января 2, 2026, https://aider.chat/docs/unified-diffs.html
26. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models, дата последнего обращения: января 2, 2026, https://arxiv.org/html/2310.05736v2
27. LLMLingua Series | Effectively Deliver Information to LLMs via Prompt Compression, дата последнего обращения: января 2, 2026, https://www.llmlingua.com/
28. Prompt Compression & LLMLingua : r/LocalLLM - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/LocalLLM/comments/1guambk/prompt_compression_llmlingua/
29. Refactoring leaderboard - Aider, дата последнего обращения: января 2, 2026, https://aider.chat/docs/leaderboards/refactor.html
30. Manage context for AI - Visual Studio Code, дата последнего обращения: января 2, 2026, https://code.visualstudio.com/docs/copilot/chat/copilot-chat-context
31. Copilot ask, edit, and agent modes: What they do and when to use them - The GitHub Blog, дата последнего обращения: января 2, 2026, https://github.blog/ai-and-ml/github-copilot/copilot-ask-edit-and-agent-modes-what-they-do-and-when-to-use-them/
32. Reducing MCP token usage by 100x — you don't need code mode - Speakeasy, дата последнего обращения: января 2, 2026, https://www.speakeasy.com/blog/how-we-reduced-token-usage-by-100x-dynamic-toolsets-v2
33. How I Reduced My AI Agent's Token Usage by 80% Using MCP Code Execution - Medium, дата последнего обращения: января 2, 2026, https://medium.com/@iamsahilshukla/how-i-reduced-my-ai-agents-token-usage-by-80-using-mcp-code-execution-94da67fe0712
34. Implementing Anthropic-style Dynamic Tool Search Tool | by Sascha Heyer | Google Cloud - Community | Nov, 2025, дата последнего обращения: января 2, 2026, https://medium.com/google-cloud/implementing-anthropic-style-dynamic-tool-search-tool-f39d02a35139
35. Code execution with MCP: building more efficient AI agents - Anthropic, дата последнего обращения: января 2, 2026, https://www.anthropic.com/engineering/code-execution-with-mcp
36. GitHub Copilot seat assignment, дата последнего обращения: января 2, 2026, https://docs.github.com/en/copilot/reference/copilot-billing/seat-assignment

37. How do you manage seat acquisition in large GitHub Enterprise accounts? - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/github/comments/1p3ybbu/how_do_you_manage_seat_acquisition_in_large/
38. AI model comparison - GitHub Docs, дата последнего обращения: января 2, 2026, https://docs.github.com/en/copilot/reference/ai-models/model-comparison
39. Do AI coding agents actually save you time, or just create more cleanup? - Reddit, дата последнего обращения: января 2, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1mdg9z1/do_ai_coding_agents_actually_save_you_time_or/